Lektion 7 · LB 3 · DS 7

Arbeitsblatt: Texte und Wörterbücher

Name:	Klasse:	Datum:

📘 **Leseauftrag:** Skript, Kapitel 7 – Texte, Wörterbücher und Verschlüsselung, Seiten 15–16
Lies: 7.1 Strings und Dictionaries – len, Index, upper(), Schleife über Zeichen und das Wörterbuch morse (S. 15) · 7.2 Morse-Code über eine Lichtstrecke – Kommunikationskanal, Takt und die Funktionen signal() und sende() (S. 15) · 7.3 Verschlüsselung – Caesar mit ord(), chr() und Modulo 26, Vigenère (S. 15) · Kasten „Am echten TXT 4.0“ (S. 15)
Hinweis: Kasten „Ausprobieren im Browser“ (das machst du in der Vertiefung) und „Teste dich“ auf S. 16 – die Lösungen in Anhang D bitte nicht nachschlagen.
Achte besonders auf: Wie greifst du auf ein einzelnes Zeichen eines Textes zu – und wie auf einen Wert im Wörterbuch? · Aus welchen drei Teilen besteht ein Kommunikationskanal? · Wie lang sind dit, dah und die Pausen – und wo entstehen sie in signal() und sende()? · Warum ist die Caesar-Chiffre leicht zu knacken?

A Verständnis – beantworte die Fragen mithilfe des Skripts

A1 text = "Kamera" – Was gibt print(text[2], len(text)) aus?

Kreuze die richtige Antwort an.

- ☐ a 6
- ☐ K 6
- ☐ m 6
- ☐ m 5

A2 Ordne die Teile der Lichtstrecke aus 7.2 zu.

Trage den passenden Buchstaben ein.

	Buchstabe
Sender	
Übertragungsweg	
Empfänger	
gemeinsamer Takt	

A = Licht · B = LED · C = Encodermotor · D = DIT = 0.1 · E = Fototransistor

A3 Bei DIT = 0.15: Wie viele Sekunden dauert sende("A") (A = .-) mit dem Programm aus 7.2 – einschließlich der Pause nach dem Buchstaben?

Rechnung:

Ergebnis: _____ s

A4 Ergänze die Funktion sende(text) aus 7.2.

for buchstabe in text.(1) _____(): macht alle Buchstaben groß. for z in (2) _____[buchstabe]: schlägt das Morsezeichen im Wörterbuch nach. Ein dah ruft signal((3) _____) auf. Nach jedem Buchstaben folgt time.sleep((4) _____) – zusammen mit der Pause aus signal() sind das 3 Einheiten.

Wähle aus: (1) split / upper / lower (2) text / rueck / morse (3) 2 * DIT / DIT / 3 * DIT (4) 7 * DIT / 2 * DIT / 3 * DIT / DIT

A5 Die Funktion soll mit der Caesar-Chiffre verschlüsseln; caesar("Zug", 2) soll BWI ausgeben. Welche Zeilen sind fehlerhaft?

Kreuze die fehlerhaften Zeilen an.

Zeile	**Fehler ?**	**Programm / Aussage**

1	☐	def caesar(text, k):
2	☐	ergebnis = ""
3	☐	for b in text.lower():
4	☐	ergebnis = ergebnis + chr((ord(b) - 65 + k) % 25 + 65)
5	☐	return ergebnis
6	☐	print(caesar("Zug", 2))

Begründung / Korrektur:

A6 Bringe die Schritte in die richtige Reihenfolge, mit denen caesar() einen Buchstaben verschlüsselt.

Nummeriere in der richtigen Reihenfolge (1 = zuerst).

Nr.	Schritt
	mit ord() in eine Zahl umwandeln (A = 65)
	Modulo 26 rechnen, damit man im Alphabet bleibt
	65 addieren und mit chr() zurück in einen Buchstaben umwandeln
	65 abziehen → Position 0 … 25
	Schlüssel k addieren
	Buchstaben groß schreiben: upper()

A7 Verschlüssele „TXT“ mit der Caesar-Chiffre und k = 5.

B Vertiefung – Simulationen und Python-Aufgaben

Die Simulationen und Python-Aufgaben findest du auf der Lernplattform. Halte deine Vermutungen, Beobachtungen und Ergebnisse hier fest. Das Arbeitsblatt ist keine Voraussetzung für den Abschluss der Lektion.

B1 Gib einen Text ein und stelle die Länge eines DIT ein. Der Zeitstrahl zeigt, wann die LED leuchtet – genau wie bei sende() aus 7.2.

Simulation auf der Lernplattform (Lektion 7, Vertiefung, Aufgabe 1)

① **Vermutung:** Wie viele Einheiten dauert sende("SOS") insgesamt – mit allen Pausen? (Vermutung)

Meine Vermutung: _____ Einheiten

② **Ausprobieren:** Sende „SOS“ und finde den DIT-Wert, bei dem die ganze Nachricht genau 4,5 s dauert.

Text	DIT (s)	Einheiten / Dauer

③ **Ergebnis erklären:**

Wie viele Sekunden dauert sende("OK") bei DIT = 0.2? (O = ---, K = -.-)

Rechnung:

Ergebnis: _____ s

B2 Schreibe zu_morse(text): jeden Buchstaben nachschlagen, die Morsezeichen mit Leerzeichen trennen. zu_morse("sos") → "... --- ...".

Python-Aufgabe auf der Lernplattform (Lektion 7)

Plane: Welche Befehle, Bedingungen und Schleifen brauchst du? Notiere die wichtigsten Python-Zeilen.

Ergebnis von ✓ Prüfen / am echten TXT:

B3 Sende das Wort mit der LED an O1. Ergänze signal(dauer) und die Schleife in sende(text).

Python-Aufgabe auf der Lernplattform (Lektion 7)

Plane: Welche Befehle, Bedingungen und Schleifen brauchst du? Notiere die wichtigsten Python-Zeilen.

Ergebnis von ✓ Prüfen / am echten TXT:

B4 Der Fototransistor des Empfängers hat diese Leuchtdauern (s) gemessen; None markiert eine Buchstabenpause. Kurz (< 0,2 s) = „.“, lang = „-“. Dekodiere die Nachricht (umgekehrtes Wörterbuch!).

Python-Aufgabe auf der Lernplattform (Lektion 7) – freiwillig

Plane: Welche Befehle, Bedingungen und Schleifen brauchst du? Notiere die wichtigsten Python-Zeilen.

Ergebnis von ✓ Prüfen / am echten TXT:

B5 Schreibe caesar(text, k) für Großbuchstaben A–Z. Tipp: (ord(b) - 65 + k) % 26 + 65. Entschlüsseln = verschlüsseln mit -k.

Python-Aufgabe auf der Lernplattform (Lektion 7)

Plane: Welche Befehle, Bedingungen und Schleifen brauchst du? Notiere die wichtigsten Python-Zeilen.

Ergebnis von ✓ Prüfen / am echten TXT:

Am echten TXT 4.0

Bearbeite am echten TXT 4.0 die Fischer-Aufgabe A5 „Morse-Code“ – am besten zu zweit mit zwei TXT: einer sendet mit der LED, der andere empfängt mit dem Fototransistor. Vorlage: 06_TXT_Programme/A5_Morse_Sender.py.

Das nimmst du mit

- Texte sind Zeichenfolgen: text[0] ist das erste Zeichen, len(text) die Länge, for z in text geht jedes Zeichen durch.
- Ein Wörterbuch ordnet einem Schlüssel einen Wert zu: morse["O"] liefert "---".
- Sender und Empfänger brauchen einen gemeinsamen Code und Takt: dit 1, dah 3, Pause im Zeichen 1, zwischen Buchstaben 3, zwischen Wörtern 7 Einheiten.
- Caesar verschiebt um k Stellen (ord, chr, % 26) – nur 25 Schlüssel, leicht zu knacken; Vigenère mit Schlüsselwort ist deutlich sicherer.